



UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

**Centro de
e-Learning**

UNIDAD DIDÁCTICA I

PYTHON PARA PRINCIPIANTES

Centro de e-Learning SCEU UTN - BA.

Medrano 951 2do piso (1179) // Tel. +54 11 4867 7589 / Fax +54 11 4032 0148

www.sceu.frba.utn.edu.ar/e-learning

Modulo I – Introducción

Unidad I – Instalación y preguntas comunes.



Presentación:

Bienvenidos a este curso de Python Paso a Paso, el mismo ha sido confeccionado como paso previo a la Diplomatura en Python de forma de introducir a aquellas personas que nunca han tenido contacto con un lenguaje de programación en los conceptos más básicos del lenguaje, desde cómo realizar la instalación, cómo utilizar un editor de texto o realizar el seguimiento de errores en el código desarrollado.

Python es un lenguaje que posee muchas aplicaciones y hoy en día se puede utilizar para realizar sitios web escalables, realizar cálculos estructurales complejos con elementos finitos, videojuegos, entre otras muchas aplicaciones. Durante este curso nos familiarizaremos con lo básico del lenguaje comenzando en esta unidad por su instalación y tratando de contestar a las preguntas más habituales que los nuevos alumnos pueden tener.



Objetivos:

Que los participantes:

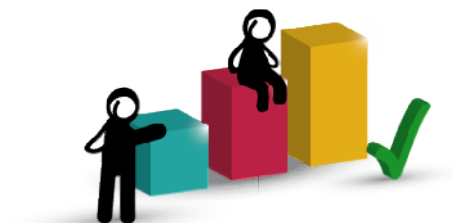
Puedan realizar la correcta instalación y puesta en funcionamiento de python.

Puedan evacuar varias de las preguntas más frecuentes que se presentan al abordar un lenguaje de programación.



Bloques temáticos:

1. ¿Qué versión de Python elijo?
2. ¿Cómo Instalo Python?
3. ¿Necesito un editor de texto en particular?
4. ¿En qué áreas es conveniente el lenguaje de Python?
5. ¿Cómo ejecuto un programa?
6. ¿Es necesario realizar todo desde cero?
7. Variables y comentarios.
8. ¿Existen nombres que no conviene utilizar al programar?
9. Operadores 1



Consignas para el aprendizaje colaborativo

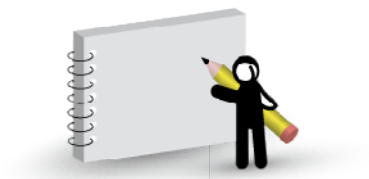
En esta Unidad los participantes se encontrarán con diferentes tipos de actividades que, en el marco de los fundamentos del MEC*, los referenciarán a tres comunidades de aprendizaje, que pondremos en funcionamiento en esta instancia de formación, a los efectos de aprovecharlas pedagógicamente:

- Los foros proactivos asociados a cada una de las unidades.
- La Web 2.0.
- Los contextos de desempeño de los participantes.

Es importante que todos los participantes realicen algunas de las actividades sugeridas y compartan en los foros los resultados obtenidos.

Además, también se propondrán reflexiones, notas especiales y vinculaciones a bibliografía y sitios web.

El carácter constructivista y colaborativo del MEC nos exige que todas las actividades realizadas por los participantes sean compartidas en los foros.



Tomen nota

Las actividades son opcionales y pueden realizarse en forma individual, pero siempre es deseable que se las realice en equipo, con la finalidad de estimular y favorecer el trabajo colaborativo y el aprendizaje entre pares. Tenga en cuenta que, si bien las actividades son opcionales, su realización es de vital importancia para el logro de los objetivos de aprendizaje de esta instancia de formación. Si su tiempo no le permite realizar todas las actividades, por lo menos realice alguna, es fundamental que lo haga. Si cada uno de los participantes realiza alguna, el foro, que es una instancia clave en este tipo de cursos, tendrá una actividad muy enriquecedora.

Asimismo, también tengan en cuenta cuando trabajen en la Web, que en ella hay de todo, cosas excelentes, muy buenas, buenas, regulares, malas y muy malas. Por eso, es necesario aplicar filtros críticos para que las investigaciones y búsquedas se encaminen a la excelencia. Si tienen dudas con alguno de los datos recolectados, no dejen de consultar al profesor-tutor. También aprovechen en el foro proactivo las opiniones de sus compañeros de curso y colegas.

1. ¿Qué versión de Python elijo?

La versión de Python que vamos a instalar es la 3.8.2 lanzada el 24 de febrero del 2020.

¿Por qué?

Simplemente porque es la última versión a la fecha y estamos estudiando.

¿Es la mejor versión?

La última versión siempre soluciona problemas que han tenido las versiones anteriores o mejoran alguna funcionalidad en un área específica. Podemos acceder a la página oficial de Python para informarnos sobre qué tipo de modificaciones se introducen en cada versión.

¿Los cambios en cada versión son grandes o pequeños?

Cada versión viene especificada con tres números separados por un punto, por ejemplo:

3.8.2

La versión anterior del 18 de Diciembre del 2019 era la:

3.8.1

Cuando se introduce una nueva versión, podemos ver que alguno de los números han variado, y el grado de importancia de los cambios introducidos aumenta de derecha a izquierda.

En el ejemplo anterior, el cambio de 1 a 2 en el número de la derecha indica que se han realizado cambios menores, que si bien es necesario revisar la documentación oficial para chequear si lo realizado afecta nuestro trabajo, podemos esperar que los cambios no sean trascendentes. Sin embargo si lo que ha variado es el primer número, por ejemplo:

2.7.1

El cambio es radical, de hecho al pasar a la versión 3.x.x. (las “x” representan cualquier número) la estructura interna de python cambio de forma tal que el programa o aplicación literalmente puede dejar de funcionar.



¿Siempre debo usar la última versión?

Esto depende de en que rama estamos trabajando, por lo general no realizamos un programa desde cero, sino que utilizamos alguna plataforma realizada por alguien (llamaremos a estas aplicaciones de aquí en más aplicaciones de terceros), estas aplicaciones a su vez es posible que utilice aplicaciones de otras personas, con lo cual cuando trabajamos debemos ver las dependencias de versiones de estas aplicaciones de terceros. Por ejemplo si queremos realizar páginas web, podemos utilizar la plataforma de Django, y al ir a la página de descarga nos informa que si queremos utilizar django con la versión de python 2.7, solo podemos utilizar la versión de Django hasta la 1.11.x, para utilizar la última versión de Django que es la 3.x.x se nos recomienda utilizar la última versión de python 3.8.x.



How to get Django

Django is available open-source under the BSD license. We recommend using the latest version of **Python 3**. The last version to support **Python 2.7** is Django 1.11 LTS. See [the FAQ](#) for the

2. ¿Cómo Instalo Python?

La versión de Python que vamos a instalar es la **3.10.11** lanzada el **5 de abril del 2023**, esta versión posee una performance mucho mayor a las anteriores y a la fecha es la última versión estable.

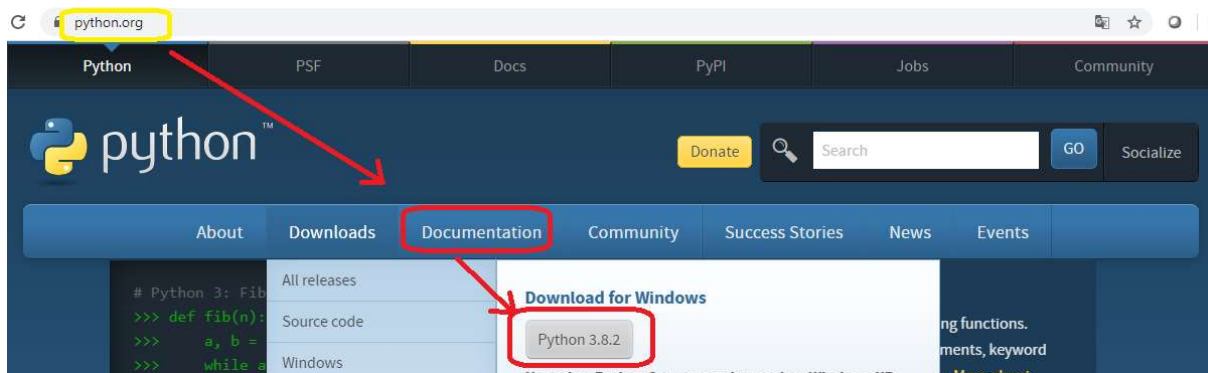
Nota: Si en el momento de leer el material el alumno encuentra una actualización, puede descargar la nueva versión y seguir los mismos pasos.

2.1. Descargar

Accedemos al sitio:

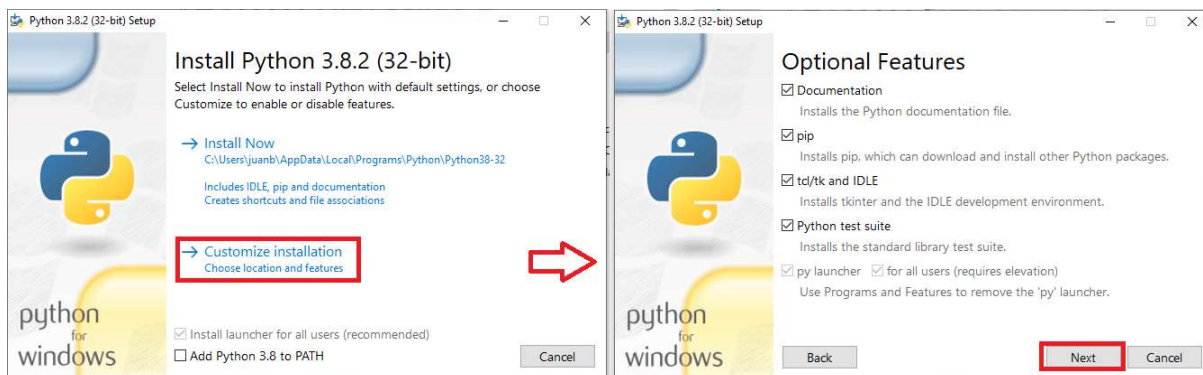
<https://www.python.org/downloads/>

Y descargamos el paquete correspondiente al sistema operativo que tenemos, en este caso para trabajar en facultad, lo haremos sobre Windows de 64bit el cual es un ejecutable.

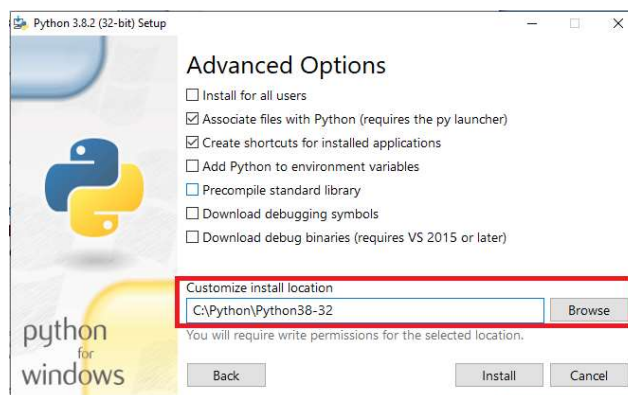


2.2. Instalación

A partir de la versión 3.5, la instalación solo requiere que presionemos en “Install Now” y que tildemos el campo de “Add Python 3.X to PATH”, aquí seleccionaremos la opción que nos permite personalizar la instalación para poder seleccionar el directorio de instalación y comprender como realizar la instalación en versiones previas. No tildaremos el campo para agregar la ruta al path.



Por defecto la opción de instalación lo instala para el usuario actual a no ser que seleccionemos que lo instale para todos los usuarios o introduzcamos la ruta donde queremos instalarlo buscándola con el botón “Browse”.



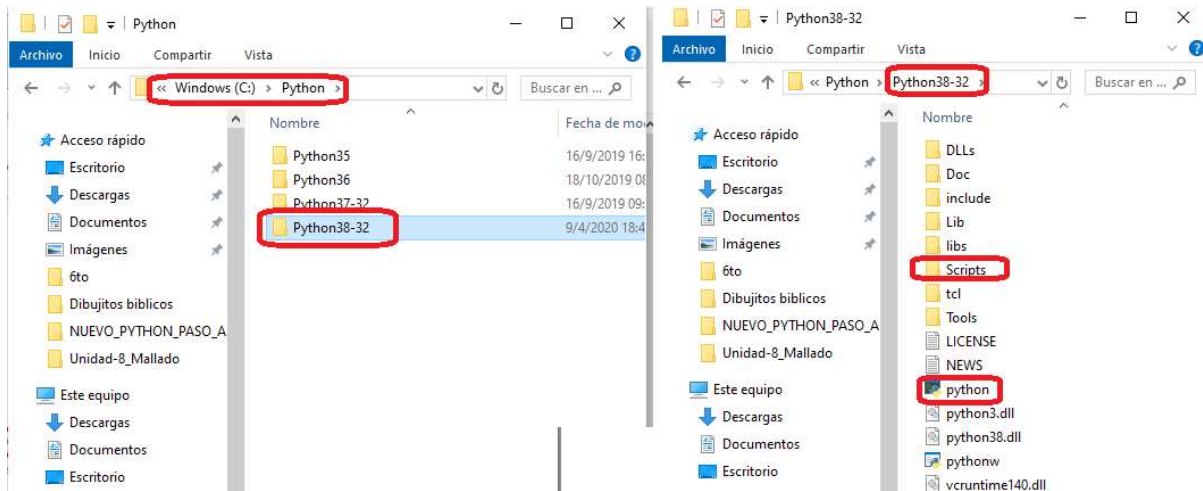
Debemos tener cuidado en el caso de seleccionar una carpeta a partir de Browse ya que si seleccionamos por ejemplo el disco “C”, agregaría todos los archivos sueltos dentro del disco. Para evitar esto indicamos el nombre de la carpeta que queremos que cree en el momento de la instalación dentro del disco para que todos los archivos se agreguen ahí dentro. Como ejemplo podríamos poner:

C:\Python38-32

Y se crearía la carpeta “Python38-32” con la finalidad de contener todos los archivos a instalar.



Finalmente presionamos “next” para comenzar la instalación, y al finalizar ya podemos ingresar a la carpeta y observar que dentro se encuentra el ejecutable de python “python.exe” y un directorio muy importante “Script” al cual aremos referencia un poco más adelante en esta unidad.



Si damos doble click sobre el ejecutable se nos abrirá una terminal desde la cual ya podemos utilizar python, a modo de ejemplo escribiremos:

Print (“Hola”)

Podremos observar cómo nos retorna “Hola”.

```
Simbolo del sistema - python
Microsoft Windows [Versión 10.0.18362.720]
(c) 2019 Microsoft Corporation. Todos los derechos reservados.

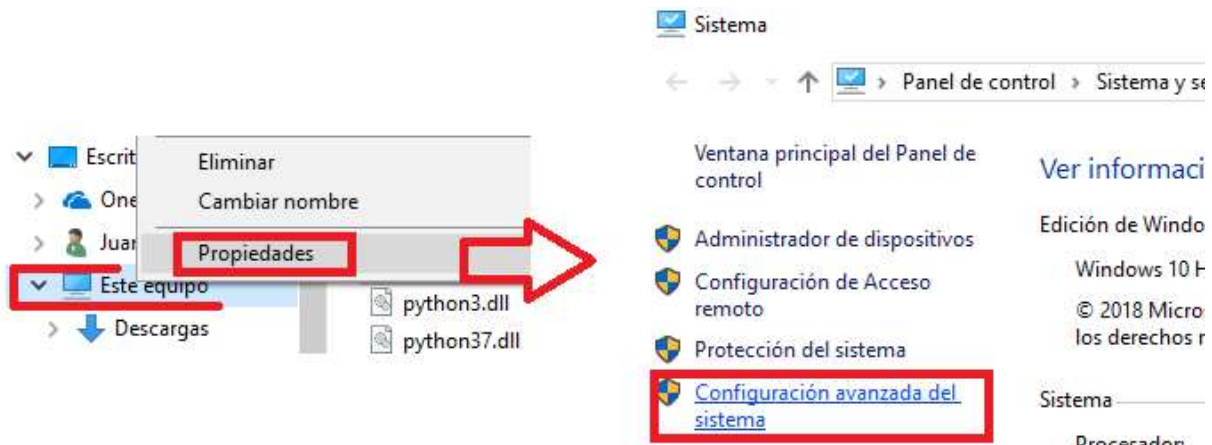
C:\Users\juanb>python
Python 3.8.2 (tags/v3.8.2:7b3ab59, Feb 25 2020, 22:45:29) [MSC v.1916 32 bit (Intel)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> print("Hola")
Hola
>>>
```

Configuración.

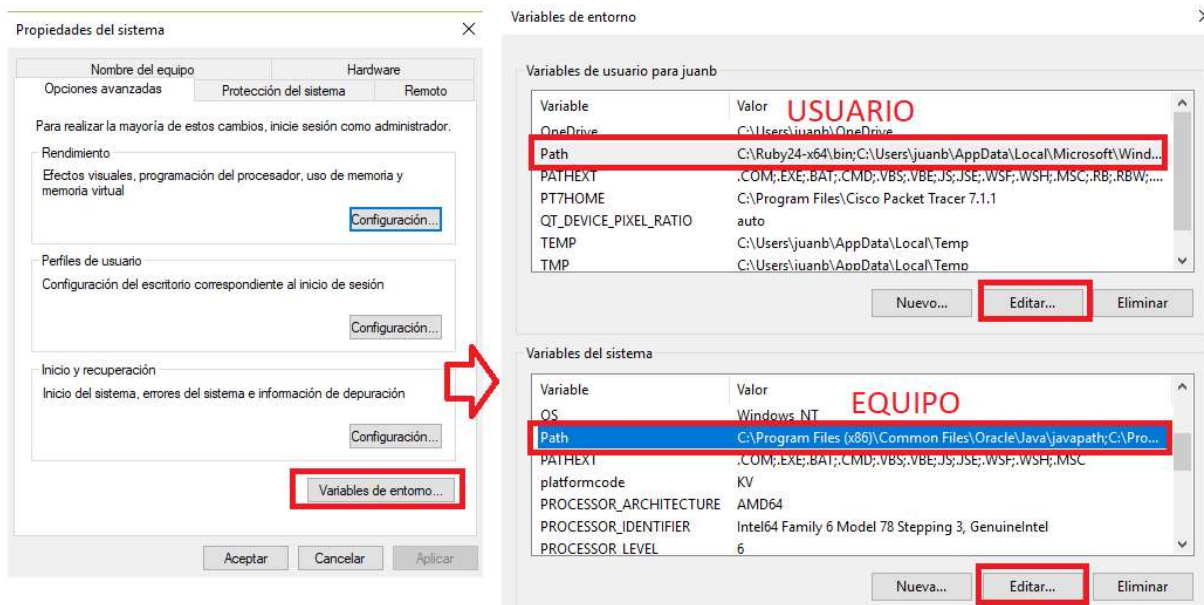
En nuestro sistema operativo, podríamos tener instalada cualquier versión de python, incluso todas ellas al mismo tiempo, sin embargo la versión que estaría ejecutándose en el sistema sería aquella que se encontrara agregada en las variables de entorno del usuario que está logueado. .

¿Cómo se agrega de forma permanente una versión de python?

Abrimos una carpeta cualquiera en Windows y hacemos click derecho sobre el ícono de “Equipo” y luego seleccionamos “Propiedades”. En la nueva ventana seleccionamos “Configuración avanzada del sistema”



Accedemos a una ventana emergente desde la cual debemos ir a “Variables de entorno” para acceder a las secciones desde las cuales poder agregar a la ruta “Path” mediante el botón “Editar”, la ruta a la versión de python que queremos utilizar ya sea para el usuario actual o para todos los usuarios del equipo.



Nota: Solo la versión de python agregada en la ruta “Path” es utilizada en el sistema operativo, por lo que cuando descargemos paquetes de terceros para extender nuestras funcionalidades de python, estos paquetes se instalarán en la versión que se encuentre indicada en la variable “Path”.

Las rutas que debemos agregar son:

C:\Python38-32
C:\Python38-32\Scripts

En Windows 10 es más fácil porque podemos agregarlas desde la ventana emergente de a una, sin embargo en versiones previas de Windows todas las rutas venían en una sola línea separadas por punto y coma (;) y debíamos agregarlas al final de la línea de esta forma:

Contenido-del-path;C:\Python38-32;C:\Python38-32\Scripts;

Nota: Debemos tener mucho cuidado de no borrar ninguna de las rutas de la variable “Path” porque podríamos generar grandes problemas en la ejecución de nuestro sistema

Centro de e-Learning SCEU UTN - BA.

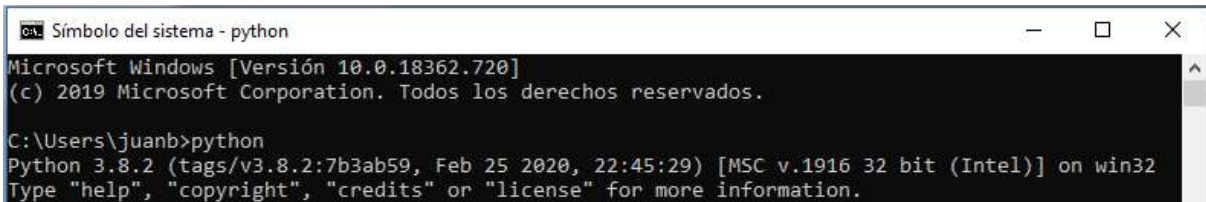
Medrano 951 2do piso (1179) // Tel. +54 11 4867 7589 / Fax +54 11 4032 0148

www.sceu.frba.utn.edu.ar/e-learning

operativo, si creemos que por accidente modificamos algo debemos cancelar la operación.

2.3. Evaluar si la instalación se realizó correctamente.

Para verificar que todo esté funcionando correctamente, abrimos el cmd y tipiamos “python”. Si todo está bien debemos ver un mensaje como el siguiente:



3. ¿Necesito un editor de texto en particular?

Este es un punto importante a aclarar, ya que una de las primeras cosas que consultan los alumnos. En general existen muchos editores que se pueden utilizar con cada lenguaje de programación, dependiendo del sistema operativo en el cual estamos trabajando, algunos son muy simples como “Ninja” y casi no requieren configuración, otros son tan complejos que son considerados más que un editor, y se los nombra como plataformas de desarrollo, como Pycharm.

¿Cuál es la diferencia?

Una aplicación como Pycharm no solo nos permite ejecutar el código de Python, sino que nos permite entre otras cosas:

- Abrir varios programas en forma simultánea.
- Poder hacer un seguimiento de los errores en el código.
- Dejar que la plataforma nos indique en colores que funcionalidad cumple cada parte del código. Etc.

Entonces ¿Siempre nos conviene usar Pycharm?

No, dependiendo del área en el que trabajamos, cada plataforma de desarrollo o editor tiene sus ventajas. Mis preferencias al trabajar son:

App	Ícono	Conveniente en:
Visual Studio Code		Al trabajar con aplicaciones en general, y en particular con: Django (Aplicaciones web)
Pycharm		Kivy (Aplicaciones multiplataformas)
Jupyter Notebook		Al realizar trabajos que queremos agregarle documentación, esta app crea una página web de cada programa y podemos agregarle texto en combinación con el código. Esta app me gusta utilizarla en el curso de: - Machine Learning, ya que viene preinstalada dentro de Anaconda (una distribución de python que viene con todas las dependencias para trabajar en el área de matemática, algebra y desarrollo de algoritmos en machine learning, big data e inteligencia artificial) - El método de los Elementos finitos, en el trabajo con la app de Fenics.
Ninja		Una app que viene con un ejecutable que detecta la distribución que estamos utilizando y nos subraya los errores de escritura del código.
Blender		Una plataforma de desarrollo en las áreas de: - Videojuegos. - Animación. - Desarrollo de texturas. - Creación de películas, etc.



IDLE		Un editor que viene con cada distribución de python, muy fácil de utilizar y con control de código, que ha incorporado a partir de la versión 3.8.x el control de líneas de código. Se ejecuta rápidamente y es muy útil para corregir código sin esperar el tiempo que tarda en abrirse una aplicación como pycharm.
------	---	---

Otras apps muy utilizadas por alumnos son: Spyder, Sublime Text, Notepad++, Eclipse, Netbeans, entre otras.

4. ¿En qué áreas es conveniente el lenguaje de Python?

En mi caso particular, Python es un lenguaje que me encanta, es un lenguaje con muy pocas restricciones en la escritura que es excelente en las áreas de:

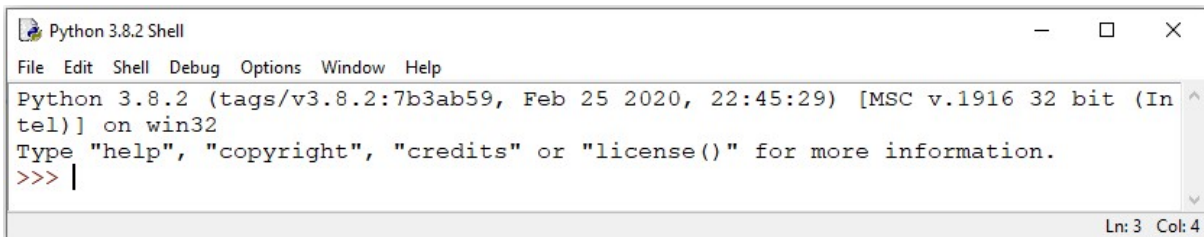
- Desarrollo de páginas web (backend y frontend)
- Machine Learning
- Inteligencia Artificial
- Big data
- Animación y desarrollo de videojuegos.
- Calculo de ecuaciones diferenciales.
- Desarrollo de algoritmos.
- Calculo por elementos finitos.

En otras áreas existen otros lenguajes que facilitan el desarrollo, no corresponde aquí comentar sobre los mismos, pero si el alumno lo requiere puede dejar una consulta en el foro de forma de recibir asesoramiento sobre cuando conviene usar otros lenguajes como: java, javascript, php, C, C#, kotlin,...

5. ¿Cómo ejecuto un programa?

Veamos ahora una pequeña aplicación, muy simple para ver como ejecutar un código. Utilizaremos en esta unidad el editor que viene con la distribución de python IDLE.

Lo abrimos y debería de presentar una apariencia como la de la siguiente imagen:



```
Python 3.8.2 Shell
File Edit Shell Debug Options Window Help
Python 3.8.2 (tags/v3.8.2:7b3ab59, Feb 25 2020, 22:45:29) [MSC v.1916 32 bit (Intel)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> |
```

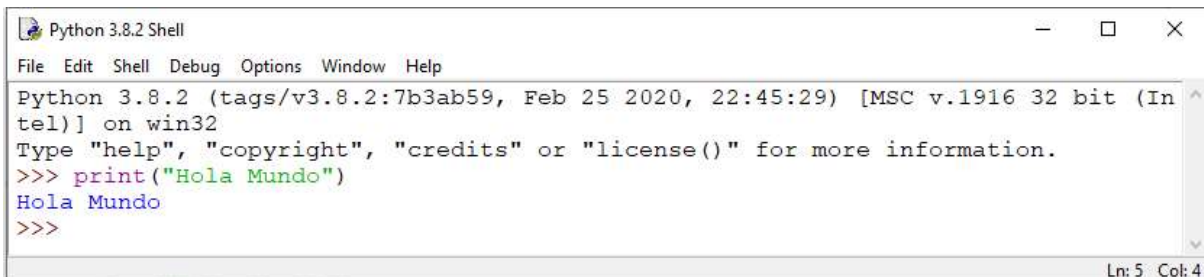
Mediante el IDLE de Python podremos evaluar el correcto funcionamiento de nuestros scripts y obtener un detalle de los posibles errores.

Como ejemplo podemos realizar nuestro primer “Hola Mundo” escribiendo:

```
print( “Hola Mundo”)
```

Notar que:

- 1.- En el código, no se agrega punto y como al final de print, como en muchos de los lenguajes de programación actuales como (php, javascript,)
- 2.- El IDLE asigna un código de colores, el cual puede ser editable.



```
Python 3.8.2 Shell
File Edit Shell Debug Options Window Help
Python 3.8.2 (tags/v3.8.2:7b3ab59, Feb 25 2020, 22:45:29) [MSC v.1916 32 bit (Intel)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> print("Hola Mundo")
Hola Mundo
>>>
```

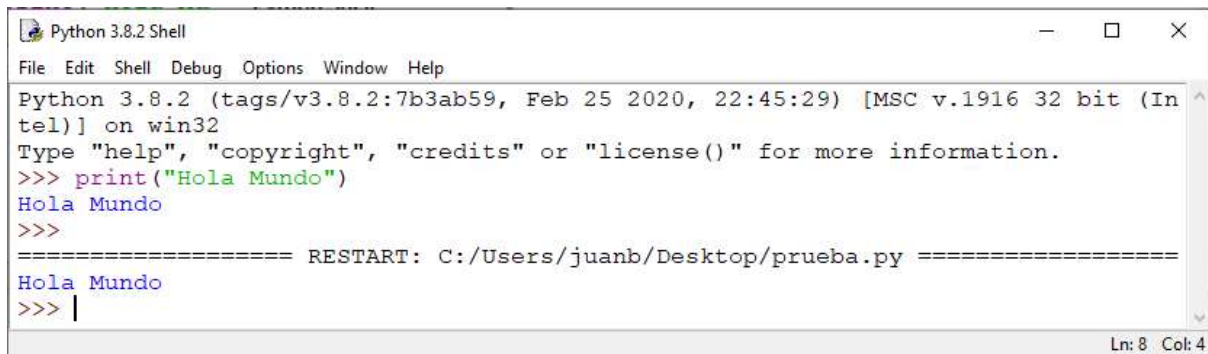


2.1.1. Crear y ejecutar script con el IDLE

Si vamos a File en el margen superior izquierdo del IDLE podemos crear un documento nuevo, al cual podemos salvar con un nombre. Si ahora dentro del archivo generado copiamos nuestro `print( "Hola Mundo")`, guardamos el archivo y vamos a Run > Run Module F5. Se abre otra ventana del IDLE que nos muestra el resultado de la ejecución y de existir errores un mensaje con el detalle de los mismos.



Visualización en la nueva ventana:



6. ¿Es necesario realizar todo desde cero?

Esta pregunta tiene una respuesta muy clara, y es NO, Python al igual que otros lenguajes de programación permite la importación de paquetes de funcionalidades y de plataformas de desarrollo que nos brindan la posibilidad de no tener que reinventar la rueda cada vez que queremos hacer algo. Por ejemplo, algunas plataformas y paquetes por área de aplicación pueden ser:

Área de aplicación						
Desarrollo web	Django	Pyramid	Bottle	Tornado	Flask	Web2py
Interfaz gráfica	tkinter	PyObject	PyQt	PySide	kivy	wxPython
Área científica	Scipy	Pandas	IPython	Numpy	math	
Machine Learning	Sklearn	Tensor Flow				

¿Cómo se importa un paquete?

Cómo veremos a continuación, cuando realicemos los ejercicios, para importar un paquete podemos utilizar las palabras reservadas **"import"** o **"from"**. Cada paquete de terceros que no se encuentre en la distribución que estamos utilizando debemos primero instalarlo.

Supongamos que estamos realizando cálculos matemáticos y necesitamos incluir el número $\pi = 3.1416\dots$, en este caso podemos utilizar la librería `math`, y de esta librería usar el número π almacenado simplemente poniendo `math.pi`, es decir de la librería `math`, utilizando notación de punto utilizamos el número π .

	ejemplo2
1	import math
2	
3	print(math.pi)

Resultado: 3.141592653589793

7. Variables y comentarios.

Variable

Una variable se puede considerar como un símbolo que puede ser reemplazado o que toma un valor determinado, como puede ser un valor numérico en una ecuación o expresión matemática en general.

Las variables se pueden utilizar para guardar datos de diferentes tipos, por ejemplo: enteros, caracteres, listas, arrays, diccionarios, objetos, etc. Todos estos términos seguramente son desconocidos a esta altura, e iremos hablando de ellos en el transcurso de las unidades.

De esta forma si queremos hacer que una variable se encuentre relacionada con un valor entero podríamos escribir algo como:

```
variable1 = 7
```

En el ejemplo anterior el nombre de la variable es "variable1" y el valor que toma la variable es el entero = 7.

Pero si el 7 lo ponemos entre comillas simples o dobles en este caso la variable1 estaría relacionada con el carácter "7" o como lo trabajaremos en python el string (alfanuméricos) "7".

```
variable1 = "7"
```

Comentarios

En python podemos adicionar a nuestro código comentarios que podemos escribir en una línea anteponiendo el símbolo de numeral "#" o en varias líneas entre comillas triples como se muestra a continuación.

```
# Esto es un comentario
```

```
'''
```

```
Hola curso,  
Esto también es un comentario pero multilínea.
```

```
'''
```



Podemos modificar el ejercicio anterior de forma de introducir una variable que luego multiplicamos por pi de la siguiente forma.

	ejemplo3
1	import math
2	
3	variable = 4 # Esta variable tiene asignado un valor entero.
4	print(variable*math.pi)

Resultado: 12.566370614359172

8. ¿Existen nombres que no conviene utilizar al programar?

Al trabajar en nuestros programas hay una serie de palabras claves que tienen un significado importante en la programación, forman parte de estructuras que utilizamos habitualmente con diferentes finalidades, por lo que no se deben utilizar fuera de su ámbito de aplicación. A continuación se deja un listado de estas palabras.

And	finally	nonlocal
As	for	not
assert	from	or
break	False	pass
class	global	raise
continue	if	return
def	import	True
del	in	try
elif	is	while
else	lambda	with
except	None	yield

9. Operadores 1

Independientemente del lenguaje de programación utilizados, es necesario contar con operadores matemáticos, los más sencillos y que todos conocemos desde el colegio suelen ser el de suma (+), resta(-), multiplicación(*) y división (/), pero existen muchos otros que pasaremos a describir a continuación separándolos en grupos.

Para los ejemplos que se presentan a continuación utilizaremos los siguientes valores de variables:

a, b, c, d, e, f = 2, 4, 5, 21, 22, 3

Notar como los valores pueden ser asignados considerando la posición que ocupan del lado izquierdo de la igualdad, con la correspondiente posición del lado derecho de la igualdad.

Grupo 1 – Operadores Aritméticos

Operador	Descripción	Ejemplo	Resultado
+	Suma	5 + 7	12
-	Resta	7 - 5	2
*	Multiplicación	2 * 3	6
/	División	21 / 3	7
%	Resto	22 % 3	1
//	División que retorna solo la parte entera	22 // 3	7
**	Potencia	4 ** 2	16

Grupo 2 – Operadores de Comparación

Operador	Descripción	Ejemplo	Resultado
>	Mayor que	print('b > a is',b>a)	b > a is True
<	Menor que	print('b < a is',b<a)	b < a is False
==	Igual a	print('b == a is',b==a)	b == a is False
!=	Distinto de	print('b != a is',b!=a)	b != a is True
>=	Mayor o igual que	print('b >= a is',b>=a)	b >= a is True
<=	Menor o igual que	print('b <= a is',b<=a)	b <= a is False



Nota: En la unidad 2 se presentaran otros grupos de operadores.



Bibliografía utilizada y sugerida

Libros

Programming Python 5th Edition – Mark Lutz – O'Reilly 2013

Programming Python 4th Edition – Mark Lutz – O'Reilly 2011

Manual online

<https://docs.python.org/3.7/tutorial/>



Lo que vimos

En esta unidad realizamos la instalación de python y comenzamos a describir su funcionamiento.



Lo que viene:

En la siguiente unidad comenzaremos a trabajar con el editor de texto Pycharm, sobre el cual realizaremos un seguimiento de los errores que se puedan presentar el código del programa. También seguiremos aprendiendo sobre otros tipos de operadores.

Centro de e-Learning SCEU UTN - BA.

Medrano 951 2do piso (1179) // Tel. +54 11 4867 7589 / Fax +54 11 4032 0148
www.sceu.frba.utn.edu.ar/e-learning